

LangChain 5-Day Intensive Bootcamp Program

Program Goal

This program is designed in a way to become productive with the LangChain ecosystem quickly. By the end of 5 days, learners should be able to:

- Build production-grade LLM applications using LangChain
 - Understand core abstractions deeply
 - Work with prompts, chains, agents, memory, RAG, tools, and observability
 - Integrate vector databases and external APIs
 - Design scalable AI workflows
 - Debug and optimize LangChain applications
 - Build multi-step AI systems for enterprise use cases
-

Pre-Requisites

Before starting the bootcamp, learners should revise:

Mandatory

- Python basics
- APIs and JSON
- Async fundamentals
- Virtual environments
- REST APIs
- Git

Recommended

- Basic understanding of LLMs
 - Prompt engineering concepts
 - Docker basics
 - FastAPI basics
-

Environment Setup

Install Python Environment

```
python -m venv venv
source venv/bin/activate
```

Install Core Packages

```
pip install langchain
pip install langchain-community
pip install langchain-openai
pip install langgraph
pip install chromadb
pip install faiss-cpu
pip install pypdf
pip install sentence-transformers
pip install tiktoken
pip install fastapi uvicorn
pip install duckduckgo-search
pip install wikipedia
pip install tavily-python
pip install python-dotenv
pip install streamlit
pip install unstructured
pip install beautifulsoup4
pip install lark
```

Recommended IDE Extensions

- Python
- Jupyter
- Ruff
- Black Formatter
- Docker

Core Learning Strategy

The program follows this structure every day:

1. Theory
2. Guided coding
3. Hands-on lab
4. Mini project
5. Reflection and debugging

DAY 1 — Foundations of LangChain

Objective

Understand the architecture, core abstractions, and ecosystem.

Topics

1. Understanding LLM Application Architecture

Concepts

- LLMs vs AI applications
- Why orchestration frameworks matter
- Stateless vs stateful workflows
- Prompt-driven systems
- Tool-augmented AI systems
- RAG architecture overview

Learning Outcome

Learners should understand where LangChain fits in modern AI architecture.

2. LangChain Ecosystem Overview

Components

- LangChain Core
- LangChain Community
- LangServe
- LangSmith
- LangGraph
- LCEL (LangChain Expression Language)

Important Design Philosophy

- Composability
 - Modularity
 - Provider abstraction
 - Runnable interface
-

3. Models in LangChain

Learn

- Chat models
- Completion models
- Embedding models
- Local vs hosted models
- OpenAI integration
- Ollama integration

Hands-On

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")
response = llm.invoke("Explain LangChain in simple words")
print(response.content)
```

4. Prompt Templates

Learn

- PromptTemplate
- ChatPromptTemplate
- System messages
- Human messages
- Placeholders
- Dynamic prompt injection

Hands-On

```
from langchain.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI architect"),
    ("human", "Explain {topic}")
])

formatted = prompt.invoke({"topic": "RAG"})
```

5. Output Parsers

Learn

- StrOutputParser
- JSON parsing
- Structured output
- Pydantic output parser

Why It Matters

Critical for enterprise-grade structured AI systems.

6. LCEL (LangChain Expression Language)

Learn

- RunnableSequence
- Pipe operators
- Chaining architecture
- Declarative pipelines

Hands-On

```
from langchain_core.output_parsers import StrOutputParser

chain = prompt | llm | StrOutputParser()
result = chain.invoke({"topic": "Agents"})
print(result)
```

Day 1 Mini Project

Project: AI Documentation Assistant

Features

- Accept topic input
- Generate technical explanation
- Generate architecture summary
- Export structured output

Skills Covered

- Prompt templates
 - LCEL
 - Chat models
 - Structured outputs
-

Recommended Learning Resources

Official Documentation

- LangChain docs
- LangGraph docs
- LangSmith docs

Videos

https://www.youtube.com/watch?v=4O1rs7mrNDo&list=PLZoTAEaLRMXVORE4VF7WQ_fAI0L1Gltar

Practice

- Build 5 prompt chains
 - Compare prompts for same task
-

DAY 2 — Chains, Memory, and Structured AI Workflows

Objective

Build advanced multi-step workflows.

Topics

1. Chains Deep Dive

Learn

- Sequential chains
- Parallel chains
- Router chains
- Conditional chains
- RunnableLambda

Architecture Discussion

How enterprise orchestration pipelines are built.

2. Memory Systems

Learn

- ConversationBufferMemory
- Window memory
- Summary memory
- Token memory
- Entity memory

Key Discussion

Why memory management is difficult in LLM systems.

3. Conversation Systems

Build

- AI chatbot
- Stateful interactions
- Context preservation
- Session-aware systems

Hands-On

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()
```

4. Structured Output and Validation

Learn

- Pydantic schemas
- Output validation
- Retry mechanisms
- Guardrails

Example

```
from pydantic import BaseModel

class UserStory(BaseModel):
    title: str
    priority: str
```

5. Error Handling and Reliability

Learn

- Retry logic
- Fallback models
- Timeout handling
- Rate limiting

- Observability patterns
-

6. Async LangChain

Learn

- ainvoke
 - batch processing
 - concurrency
 - streaming responses
-

Day 2 Mini Project

Project: AI Requirements Analyzer

Features

- Upload requirement text
- Extract user stories
- Generate tasks
- Prioritize requirements
- Maintain conversation context

Skills Covered

- Memory
 - Structured output
 - Chains
 - Async execution
-

Advanced Practice

Build

- Multi-step workflow
 - Retry-enabled pipeline
 - Stateful chatbot
-

DAY 3 — Retrieval-Augmented Generation (RAG)

Objective

Master enterprise-grade RAG systems.

Topics

1. Why RAG Matters

Learn

- Hallucination problem
- Knowledge grounding
- Enterprise document search
- Context injection

Architecture

- Loader
 - Splitter
 - Embedding
 - Vector store
 - Retriever
 - Generator
-

2. Document Loaders

Learn

- PDFLoader
- CSVLoader
- WebBaseLoader
- DirectoryLoader

Hands-On

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample.pdf")
docs = loader.load()
```

3. Text Splitting

Learn

- RecursiveCharacterTextSplitter
- Chunk size strategies
- Overlap tuning
- Semantic chunking

Critical Discussion

Chunking quality directly impacts RAG quality.

4. Embeddings

Learn

- OpenAI embeddings
 - Sentence transformers
 - Embedding dimensions
 - Semantic similarity
-

5. Vector Databases

Learn

- ChromaDB
- FAISS
- Pinecone
- Weaviate

Hands-On

```
from langchain_community.vectorstores import Chroma
```

6. Retrievers

Learn

- Similarity search
 - MMR retrieval
 - Hybrid retrieval
 - Self-query retriever
 - Multi-query retriever
-

7. Building RAG Pipelines

Learn

- RetrievalQA
 - Context injection
 - Source citation
 - Retrieval evaluation
-

8. RAG Optimization

Advanced Topics

- Re-ranking
 - Compression retrievers
 - Context window optimization
 - Parent-child retrieval
 - Metadata filtering
-

Day 3 Mini Project

Project: Enterprise Knowledge Assistant

Features

- Upload PDFs
- Semantic search
- Ask questions
- Cite sources
- Multi-document retrieval

Skills Covered

- RAG
 - Vector DB
 - Retrieval optimization
 - Context management
-

Suggested Challenge

Compare

- Small chunks vs large chunks
- Different embedding models
- Different retrievers

Measure quality differences.

DAY 4 — Agents, Tools, and Autonomous AI Systems

Objective

Build intelligent autonomous AI systems.

Topics

1. Understanding Agents

Learn

- Agentic workflows
- ReAct architecture
- Tool calling
- Planning vs execution

Critical Concept

Agents are reasoning systems, not just prompt pipelines.

2. Tools in LangChain

Learn

- Built-in tools
- Custom tools
- API integration
- Function calling

Hands-On

```
from langchain.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    return a * b
```

3. Agent Types

Learn

- Zero-shot agents
 - OpenAI tools agent
 - Structured chat agents
 - Multi-agent systems
-

4. External Integrations

Integrate

- Search APIs
 - Wikipedia
 - REST APIs
 - SQL databases
 - Python REPL
-

5. LangGraph Introduction

Learn

- Stateful graph execution
- Node-based orchestration
- Cyclic workflows
- Durable execution

Why LangGraph Matters

Modern enterprise AI systems increasingly use graph-based orchestration.

6. Multi-Agent Architecture

Learn

- Supervisor agents
 - Worker agents
 - Task delegation
 - Collaboration patterns
-

7. Human-in-the-Loop Systems

Learn

- Approval workflows
 - Escalation systems
 - Safe execution
 - Governance
-

Day 4 Mini Project

Project: Autonomous Research Agent

Features

- Accept research topic
- Search web
- Summarize findings
- Generate structured report
- Use multiple tools

Skills Covered

- Agents
 - Tool calling
 - LangGraph
 - Multi-step reasoning
-

Advanced Challenge

Build

A multi-agent architecture:

- Planner agent
 - Research agent
 - Summarizer agent
 - QA validator agent
-

DAY 5 — Production-Grade AI Systems

Objective

Learn enterprise deployment and architecture best practices.

Topics

1. Production Architecture

Learn

- Stateless APIs
- Session management
- Caching
- Scaling patterns
- Cost optimization

2. LangServe

Learn

- Serving LangChain apps
- API exposure
- Streaming APIs
- Remote invocation

Hands-On

```
from fastapi import FastAPI
```

3. Observability with LangSmith

Learn

- Tracing
- Debugging
- Prompt inspection
- Token usage tracking
- Latency analysis

Critical Enterprise Skill

Without observability, AI systems become impossible to maintain.

4. Evaluation Frameworks

Learn

- RAG evaluation
 - Hallucination detection
 - Human evaluation
 - Automated scoring
-

5. Security and Governance

Learn

- Prompt injection
 - Data leakage
 - PII masking
 - Guardrails
 - Output validation
-

6. Cost Optimization

Learn

- Token reduction
 - Caching embeddings
 - Efficient retrieval
 - Smaller model usage
 - Batch processing
-

7. Deployment Strategies

Learn

- Docker deployment
 - Kubernetes basics
 - CI/CD for AI systems
 - Cloud deployment
-

8. Future of LangChain Ecosystem

Learn

- LangGraph adoption
- AI agents trend
- MCP architecture

- Structured reasoning
 - Workflow orchestration
-

Final Capstone Project

Enterprise AI Assistant Platform

Objective

Build a production-style AI platform.

Features

1. Document Upload

- PDF ingestion
- Text extraction
- Metadata tagging

2. RAG Search

- Semantic retrieval
- Source citation
- Multi-document support

3. Agent Workflow

- Research agent
- Summarization agent
- QA validation agent

4. API Layer

- FastAPI backend
- REST endpoints
- Streaming support

5. Frontend

- Streamlit UI
- Chat interface
- Upload interface

6. Observability

- LangSmith tracing
 - Logging
 - Error handling
-

Suggested Folder Structure

```
project/
|
├─ app/
├─ agents/
├─ chains/
├─ prompts/
├─ retrievers/
├─ tools/
├─ vectorstore/
├─ api/
├─ ui/
├─ tests/
├─ config/
└─ main.py
```

Enterprise Best Practices

Architecture Principles

1. Keep prompts versioned
 2. Separate orchestration from business logic
 3. Never tightly couple model providers
 4. Build observability from day one
 5. Evaluate continuously
 6. Prefer structured outputs
 7. Optimize retrieval before upgrading models
 8. Use smaller models whenever possible
-

Common Mistakes to Avoid

Technical Mistakes

- Overusing agents
 - Poor chunking strategy
 - No evaluation framework
 - Huge prompts
 - Ignoring token costs
 - Lack of retries
 - No observability
 - Tight coupling to OpenAI
-

Deliverables Expected:

- Build RAG applications
 - Build AI agents
 - Create production-ready AI APIs
 - Design LangGraph workflows
 - Integrate external tools
 - Implement memory systems
 - Optimize token usage
 - Debug LangChain applications
 - Deploy AI systems
 - Build enterprise AI assistants
-

Suggested Daily Schedule

Daily Timeline

Activity	Duration
Theory	2 Hours
Coding	3 Hours
Review and Debugging	2 Hour

Total: ~ 7 Hours/Day

Advice:

LangChain should not be treated as just a framework.

It is an orchestration layer for building intelligent systems.

The most successful engineers:

- Focus on architecture
- Design reliable workflows
- Build strong retrieval systems
- Understand observability
- Optimize for maintainability
- Use agents only where necessary

Strong AI engineering is ultimately about system design, not only prompting.

Exercises

Exercise 1

Build a Resume Screening Assistant.

Exercise 2

Build a SQL Agent.

Exercise 3

Build a Customer Support RAG Bot.

Exercise 4

Build a Code Review Assistant.

Exercise 5

Build a Meeting Notes Summarizer.

Final Outcome

After completing this 5-day intensive program, engineers should have enough practical exposure to:

- Design enterprise AI systems
- Build production-grade LangChain applications
- Architect agentic workflows
- Implement RAG effectively
- Integrate modern LLM infrastructure
- Evaluate and optimize AI applications

This learning path is intentionally practical and architecture-focused to align with real-world enterprise AI development.